```
# STEP 1: IMPORT LIBRARIES
import pandas as pd
import numpy as np
from numpy import log, dot, exp, shape
from sklearn.metrics import confusion_matrix, accuracy_score


# STEP 2: LOAD DATASET
# Upload suv_data.csv in Colab
from google.colab import files
uploaded = files.upload()

data = pd.read_csv('Social_Network_Ads.csv')

print("Dataset Preview:\n", data.head())


# STEP 3: PREPARE DATA
# Independent variables (Age, EstimatedSalary)
x = data.iloc[:, [2, 3]].values

# Dependent variable (Purchased)
y = data.iloc[:, 4].values


# STEP 4: TRAIN TEST SPLIT
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.10, random_state=0
)


# STEP 5: FEATURE SCALING (INBUILT)
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)

print("\nSample Scaled Data:\n", x_train[:5])


# STEP 6: LOGISTIC REGRESSION (INBUILT)
from sklearn.linear_model import LogisticRegression

classifier = LogisticRegression(random_state=0)
classifier.fit(x_train, y_train)

# Prediction
y_pred = classifier.predict(x_test)

print("\nPredicted Values:\n", y_pred)


# STEP 7: EVALUATION (INBUILT)
cm = confusion_matrix(y_test, y_pred)
```

```
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

print("Accuracy:", accuracy_score(y_test, y_pred))


# STEP 8: USER DEFINED STANDARDIZATION
def standardize(X):
    for i in range(shape(X)[1]):
        X[:, i] = (X[:, i] - np.mean(X[:, i])) / np.std(X[:, i])


# STEP 9: USER DEFINED F1 SCORE
def F1_score(y, y_hat):
    tp = tn = fp = fn = 0

    for i in range(len(y)):
        if y[i] == 1 and y_hat[i] == 1:
            tp += 1
        elif y[i] == 1 and y_hat[i] == 0:
            fn += 1
        elif y[i] == 0 and y_hat[i] == 1:
            fp += 1
        elif y[i] == 0 and y_hat[i] == 0:
            tn += 1

    precision = tp / (tp + fp + 1e-10)
    recall = tp / (tp + fn + 1e-10)

    return 2 * precision * recall / (precision + recall + 1e-10)


# STEP 10: USER DEFINED LOGISTIC REGRESSION
class MyLogisticRegression:

    def sigmoid(self, z):
        return 1 / (1 + exp(-z))

    def initialize(self, X):
        weights = np.zeros((shape(X)[1] + 1, 1))
        X = np.c_[np.ones((shape(X)[0], 1)), X]
        return weights, X

    def fit(self, X, y, alpha=0.001, iterations=400):
        weights, X = self.initialize(X)
        y = y.reshape(-1, 1)

        for i in range(iterations):
            z = dot(X, weights)
            predictions = self.sigmoid(z)

            weights = weights - alpha * dot(X.T, (predictions - y))

        self.weights = weights

    def predict(self, X):
        _, X = self.initialize(X)
        z = dot(X, self.weights)
        probs = self.sigmoid(z)
```

```
        return [1 if i > 0.5 else 0 for i in probs]


# STEP 11: TRAIN USER DEFINED MODEL
# Standardize manually
standardize(x_train)
standardize(x_test)

model = MyLogisticRegression()
model.fit(x_train, y_train)

# Predictions
y_pred_ud = model.predict(x_test)
y_train_pred = model.predict(x_train)


# STEP 12: EVALUATION (USER DEFINED)
f1_train = F1_score(y_train, y_train_pred)
f1_test = F1_score(y_test, y_pred_ud)

print("\nF1 Score (Train):", f1_train)
print("F1 Score (Test):", f1_test)

# Confusion Matrix
conf_mat = confusion_matrix(y_test, y_pred_ud)
accuracy = (conf_mat[0, 0] + conf_mat[1, 1]) / np.sum(conf_mat)

print("Confusion Matrix:\n", conf_mat)
print("Accuracy:", accuracy)


print('\n\n\nProgram Executed')
```

Choose Files Social_Network_Ads.csv

Social_Network_Ads.csv(text/csv) - 10926 bytes, last modified: 4/11/2026 - 100% done

```
Saving Social_Network_Ads.csv to Social_Network_Ads (4).csv
Dataset Preview:
     User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510    Male   19            19000          0
1  15810944    Male   35            20000          0
2  15668575  Female   26            43000          0
3  15603246  Female   27            57000          0
4  15804002    Male   19            76000          0

Sample Scaled Data:
 [[-1.05714987  0.53420426]
 [ 0.2798728  -0.51764734]
 [-1.05714987  0.41733186]
 [-0.29313691 -1.45262654]
 [ 0.47087604  1.23543867]]

Predicted Values:
 [0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 1 0 0 0 0 0 0 1 0 0 0 0
 0 0 1]

Confusion Matrix:
 [[31  1]
 [ 1  7]]
Accuracy: 0.95

F1 Score (Train): 0.7649402389935018
F1 Score (Test): 0.7619047618503402
Confusion Matrix:
 [[27  5]
 [ 0  8]]
Accuracy: 0.875
```